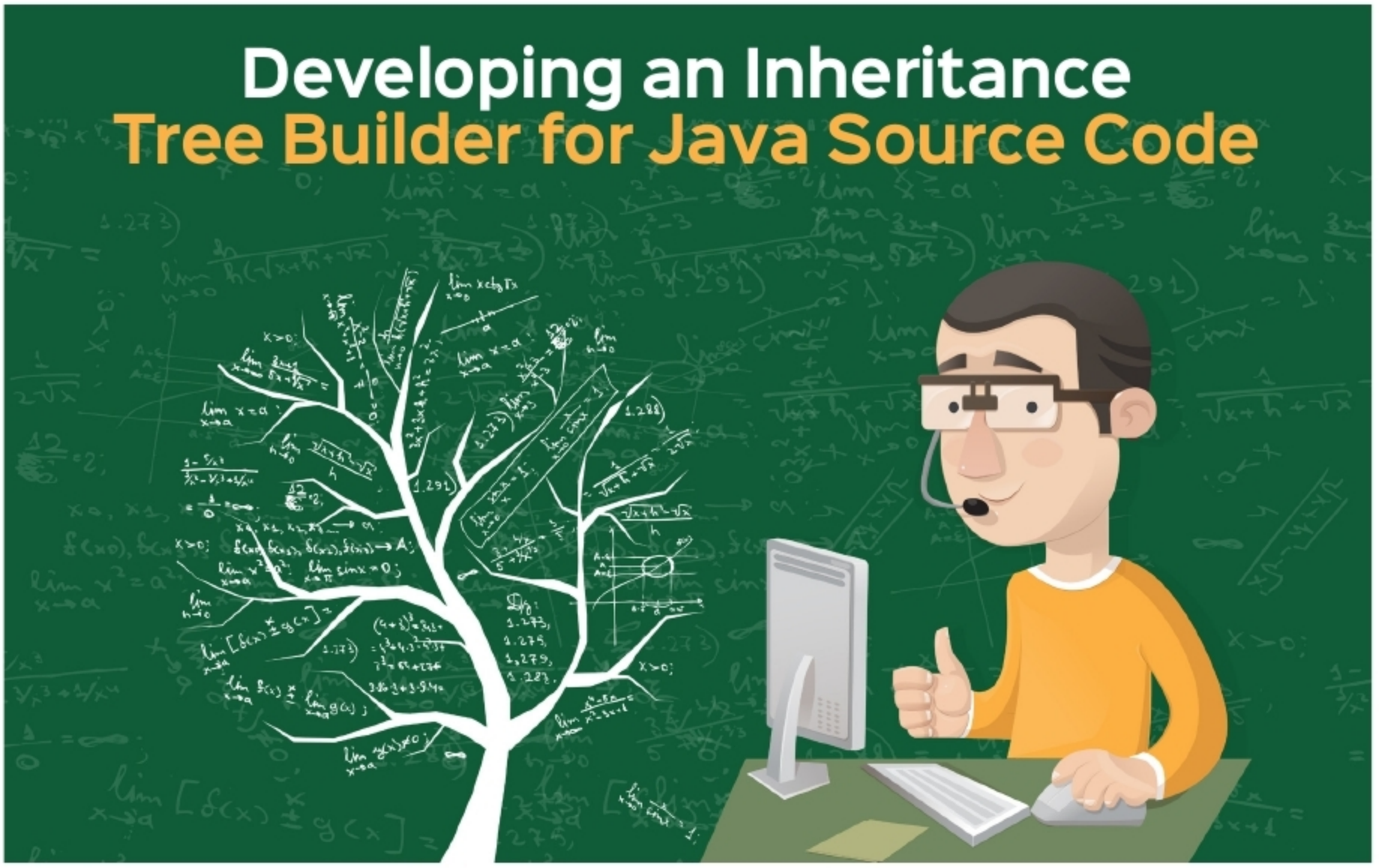
Developing an Inheritance Tree Builder for Java Source Code



Developing any application usually involves multiple developers. To understand what the others have coded, a developer needs to put in a lot of effort. To make it easier for the developer to understand complex project code, this article introduces a tool that builds the inheritance tree in the input Java package.

In object-oriented programming, inheritance is a mechanism by which a child class can inherit the properties of the parent class. In general, app development evolves across various stages, which are shown in Figure 1.

Code flows across most of the phases/stages shown in this figure. Various different teams tend to work on each stage. Visualising all the classes and their relations in a package such as the list of all classes, inherited classes, inner classes, etc, is a time consuming process (which may be due to a large code base). To avoid wasting the developers' time and to improve their efficiency, there is now a tool, called the Inheritance Tree Builder, that builds a GUI of the inheritance tree when a Java package is given as the input.

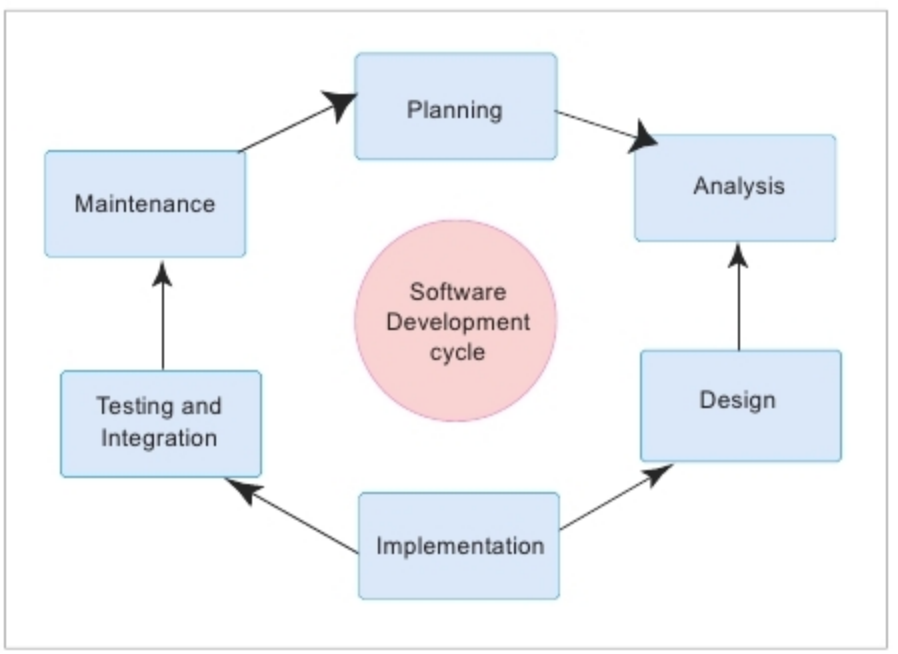


Figure 1: Software development cycle

The stages involved in the building of the tool are:

1. Listing out all the classes in a package
2. Listing out all inner class-outer class relations in a package
3. Listing out inheritance relations in a package
4. Visualising the above listed classes, the inner classes and the inherited classes in the form of a tree structure

To build the tool, the Java parser is used to get information about the input source code.

What is a Java parser?

A parser takes a sequence of tokens or program instructions as input and builds a tree-like data structure called the Abstract Syntax Tree (AST).

When we compile code, the input source code is first passed to the lexer, which outputs a list of tokens (a token is the smallest meaningful element of the program). This is passed as an input to the Syntactic Analysis stage (parser),